Programowanie obiektowe i inżynieria oprogramowania

Lab 9 - GIT:
- nowe gałęzie projektu,
- git branch / checkout / rebase

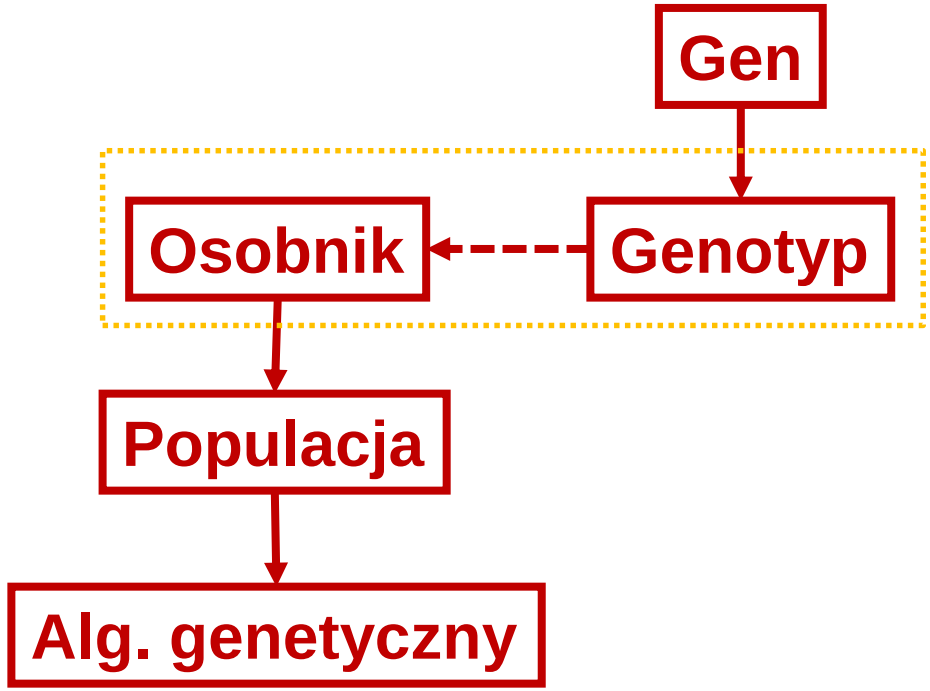
link do najnowszej wersji instrukcji

Akademia Górniczo-Hutnicza im. Stanisława Staszica w Krakowie
AGH University of Science and Technology

Prowadzący: dr inż. Jakub Bryła
2024

Symulacja implementacji algorytmu genetycznego z GIT

W pierwszej części instrukcji podejmiemy próbę zasymulowania kolejnych kroków implementacji algorytmu genetycznego – tak jak będziemy to robić na zajęciach



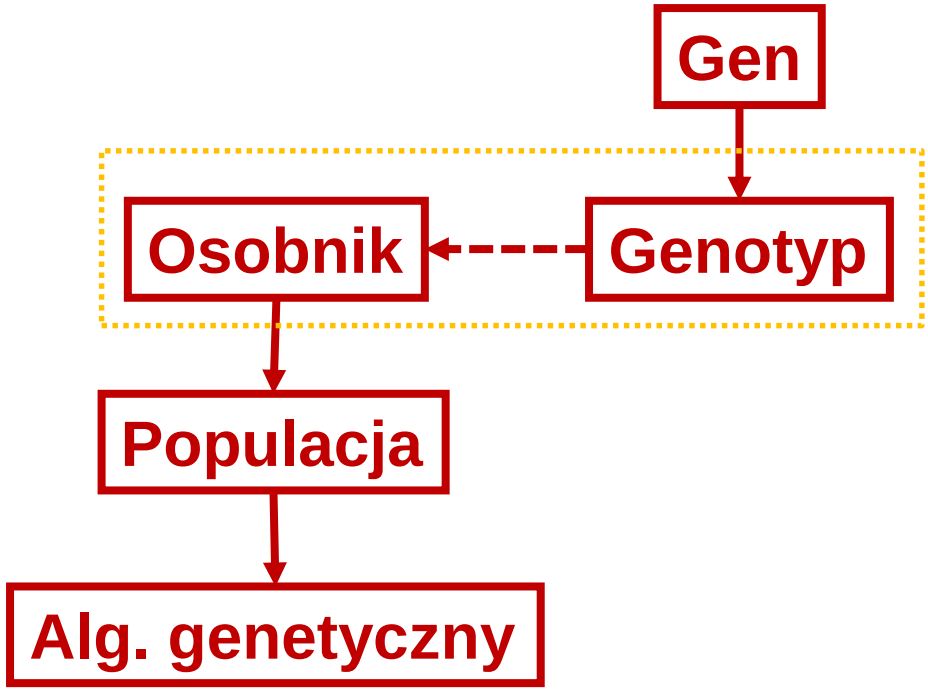
1. Tworzymy nowe repozytorium *alg_gen_txt*.
2. Tworzymy plik TGen.txt z zawartością „*Kod klasy TGen.*”. Commit.
3. Tworzymy plik TCandidate.txt z zawartością „*Kod klasy TCandidate.*”. Commit.
4. Tworzymy plik TPopulation.txt z zawartością „*Kod klasy TPopulation.*”. Commit.
5. Tworzymy plik TAlgorithm.txt z zawartością:

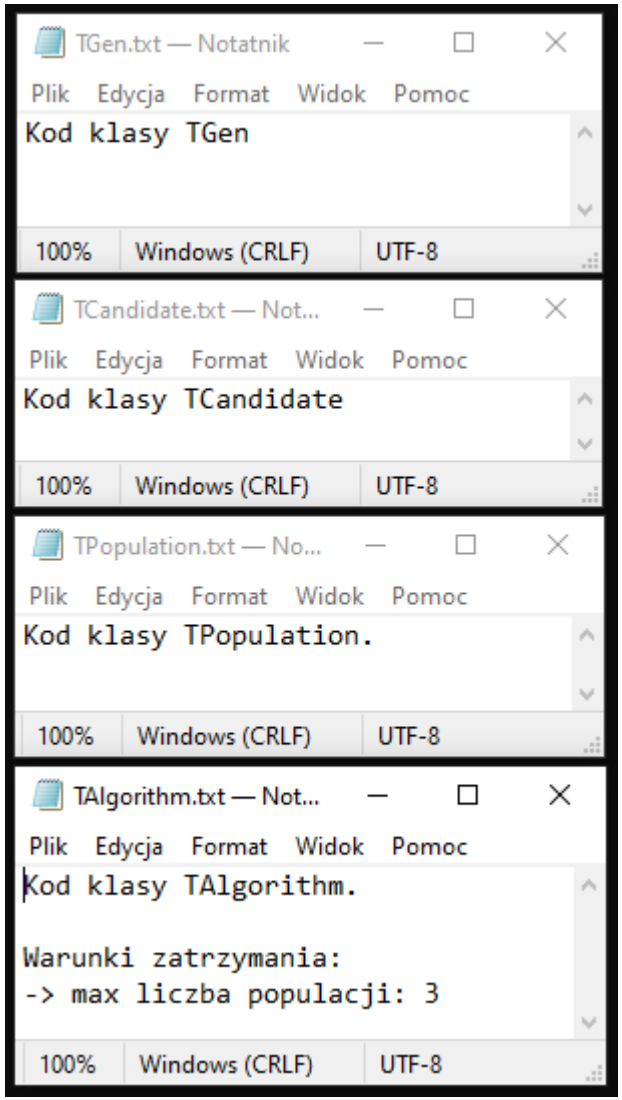
„*Kod klasy TAlgorithm.*

Warunki zatrzymania:
-> max liczba populacji: 3”.
Commit.

(wynik przedstawiony na kolejnym slajdzie)

```
>> git init alg_gen_txt
>> cd alg_gen_txt

>> notepad Tgen.txt
>> git add Tgen.txt
>> git commit –m „Klasa TGen”

>> notepad TCandidate.txt
>> git add TCandidate.txt
>> git commit –m „Klasa TCandidate”

>> notepad TPopulation.txt
>> git add TPopulation.txt
>> git commit –m „Klasa TPopulation”

>> notepad TAlgorithm.txt
>> git add TAlgorithm.txt
>> git commit –m „Klasa TAlgorithm”
```





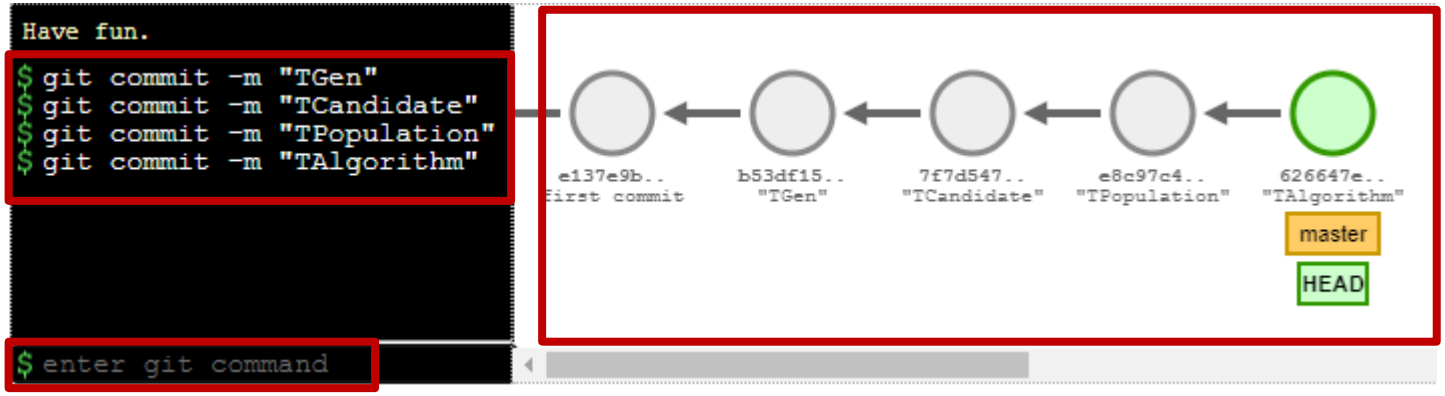
Obecny stan repozytorium

```
>> git log --oneline
```

Dla celów poznawczych zwizualizujemy stan repozytorium.

https://onlywei.github.io/explain-git-with-d3/#zen

historia wywołanych komend



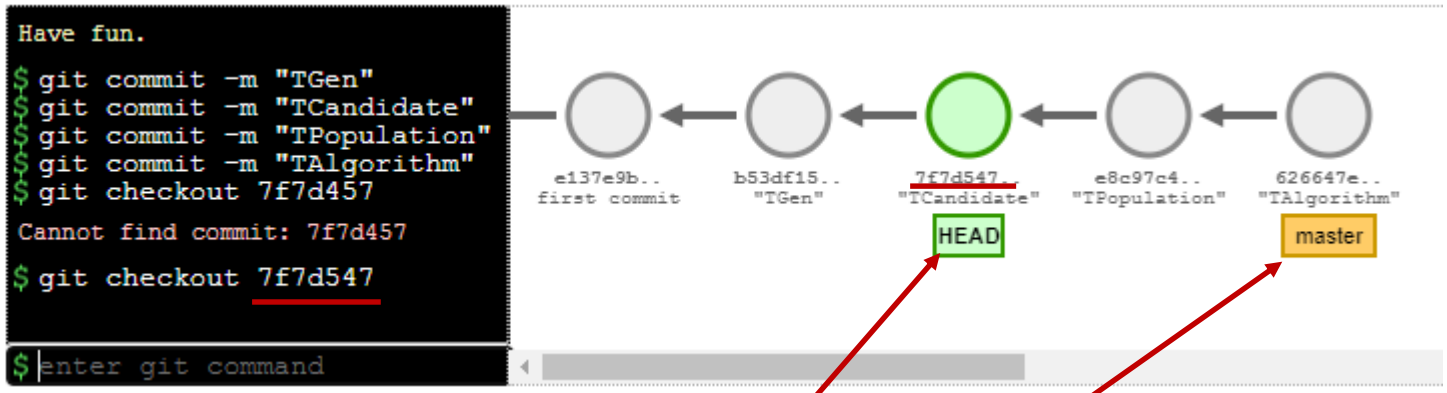
wizualizacja stanu repozytorium

pole do wpisywania **wybranych** komend GIT

Przegląd stanu projektu w historii repozytorium

Za pomocą polecenia „git checkout <*id*>” możemy wrócić do wcześniejszego stanu projektu – zapisanego w commit o podanym id.

Wizualizacja:



W <*id*> podajemy pierwsze znaki z 40-znakowego kodu referencyjnego dla interesującego nas commita

Master to etykieta – nazwa gałęzi. O tym później.
Head wskazuje obecnie wczytany commit.

Zawartość katalogu projektu / repozytorium

obecny stan projektu

```
>> git log --oneline
114d9e3 (HEAD -> master) Klasa TAlgorithm
ecc6a73 Klasa TPopulation
0f73f90 Klasa TCandidate
e02b299 Klasa TGen
```

Nazwa
- .git
- TAlgorithm.txt
- TCandidate.txt
- TGen.txt
- TPopulation.txt

stan projektu z drugiego commita

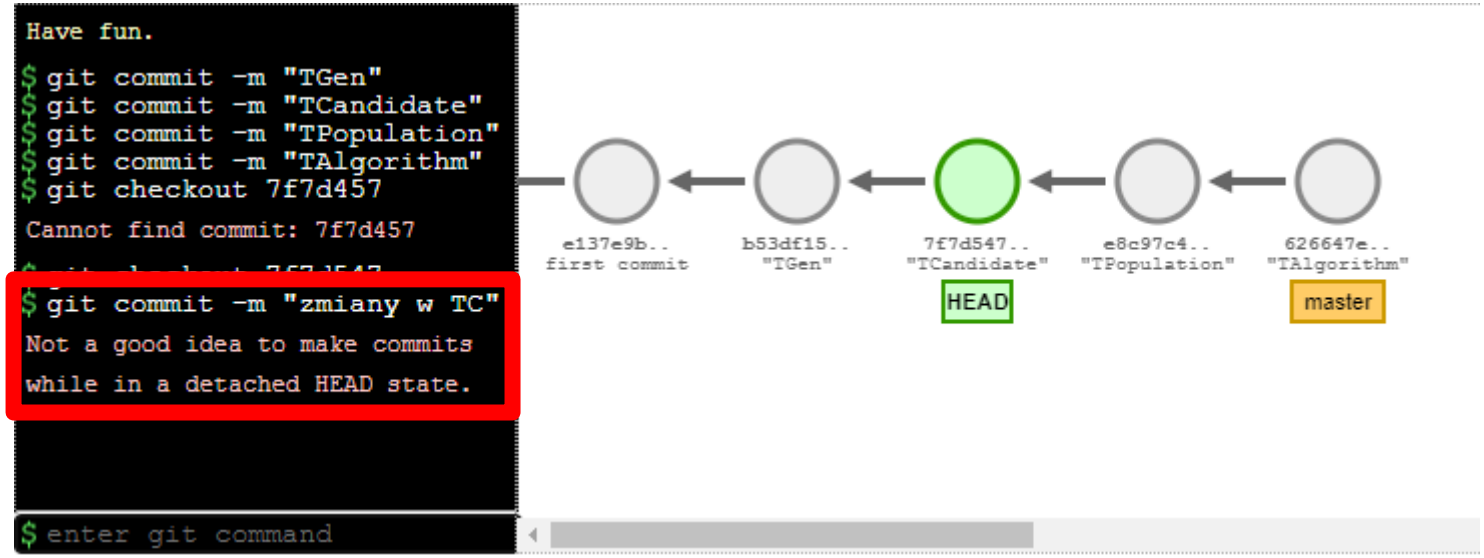
```
>> git checkout 0f73f90
>> 0f73f90 (HEAD) Klasa TCandidate
e02b299 Klasa TGen
```

Nazwa
- .git
- TCandidate.txt
- TGen.txt

powrót do obecnego stanu projektu

```
>> git checkout master
>> git log --oneline
114d9e3 (HEAD -> master) Klasa TAlgorithm
ecc6a73 Klasa TPopulation
0f73f90 Klasa TCandidate
e02b299 Klasa TGen
```

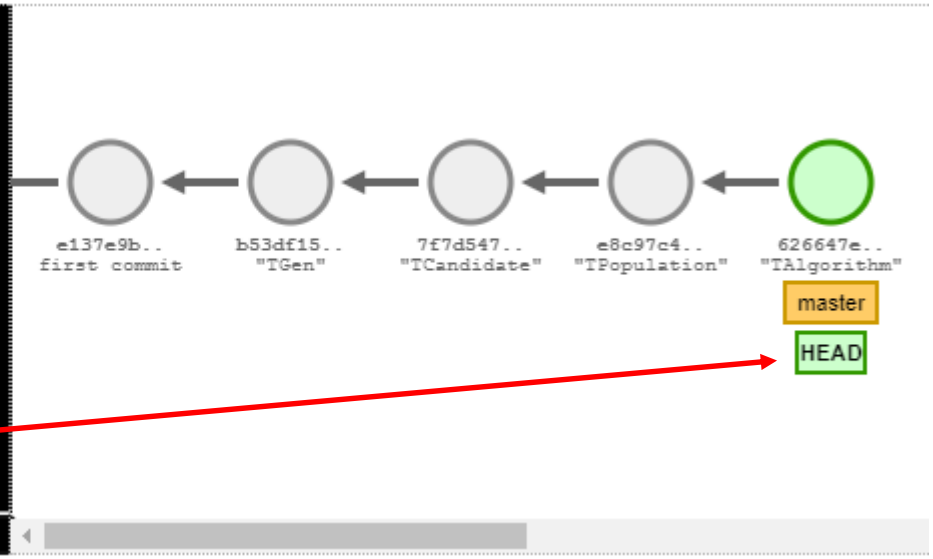
Nazwa
- .git
- TAlgorithm.txt
- TCandidate.txt
- TGen.txt
- TPopulation.txt



Po wpisaniu komendy *git checkout <id>* znajdujemy się w trybie „detached head” – znacznik *head* znajduje się gdzieś w środku gałęzi *master*. Oznacza to, że wszelkie wprowadzone zmiany i wywołanie komendy *commit* nie spełnią swojego zadania. **W przyszłości utracimy wprowadzone w ten sposób zmiany – GIT usunie je atomatycznie.**

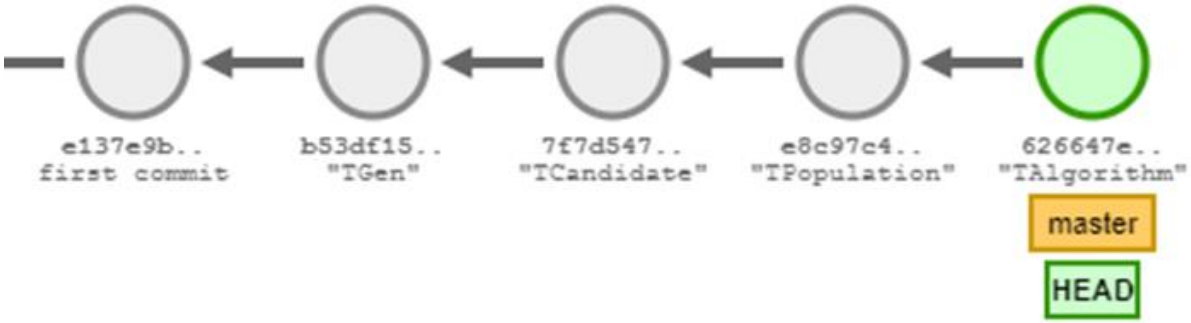
Powrót do ostatniego *commita*:

```
Have fun.

$ git commit -m "TGen"
$ git commit -m "TCandidate"
$ git commit -m "TPopulation"
$ git commit -m "TAlgorithm"
$ git checkout 7f7d457

Cannot find commit: 7f7d457

$ git checkout 7f7d547
$ git commit -m "zmiany w TC"

Not a good idea to make commits

while in a detached HEAD state.

$ git checkout master

$ enter git command
```

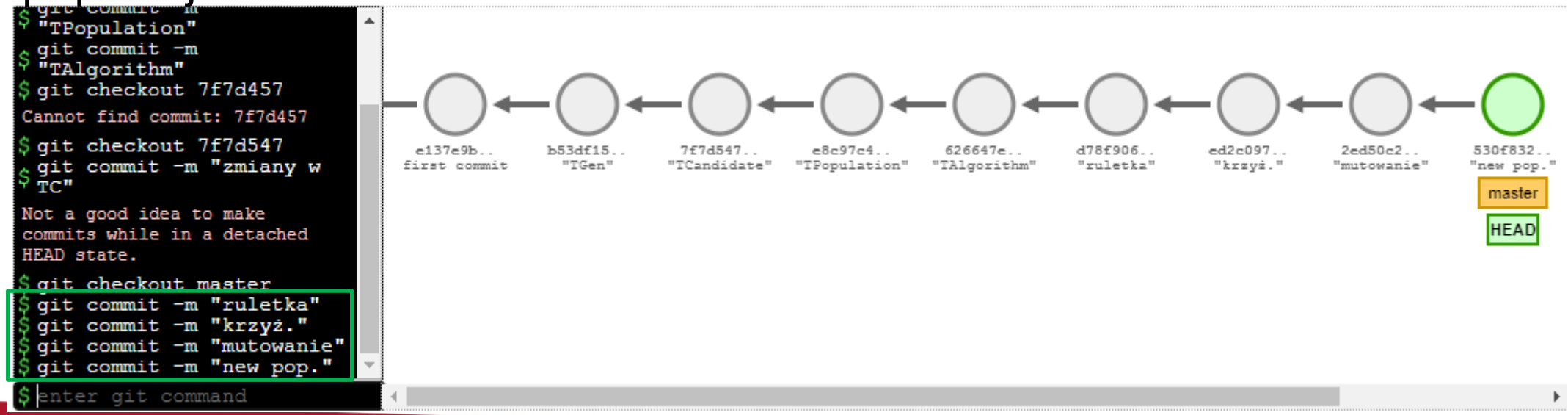


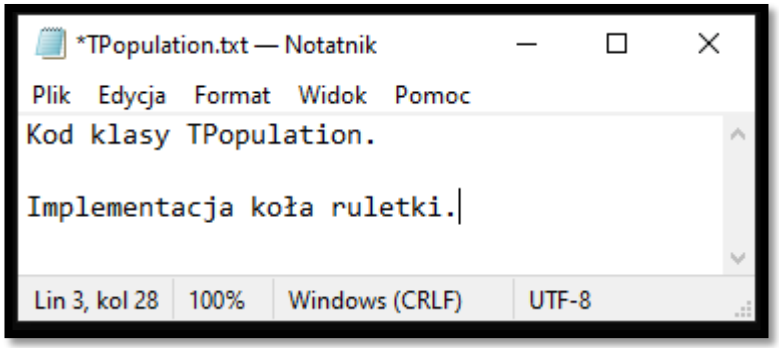
Dalsze prace nad projektem

Obecny stan projektu:

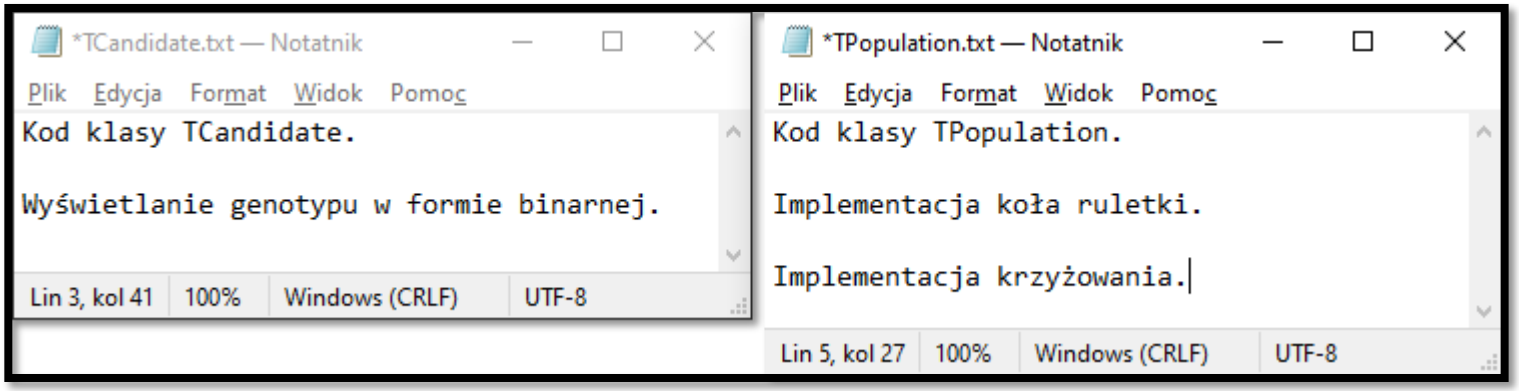


Dalsze prace nad algorytmem genetycznym polegają na zaimplementowaniu funkcjonalności koła ruletki, krzyżowania, mutowania oraz tworzenia nowej populacji:

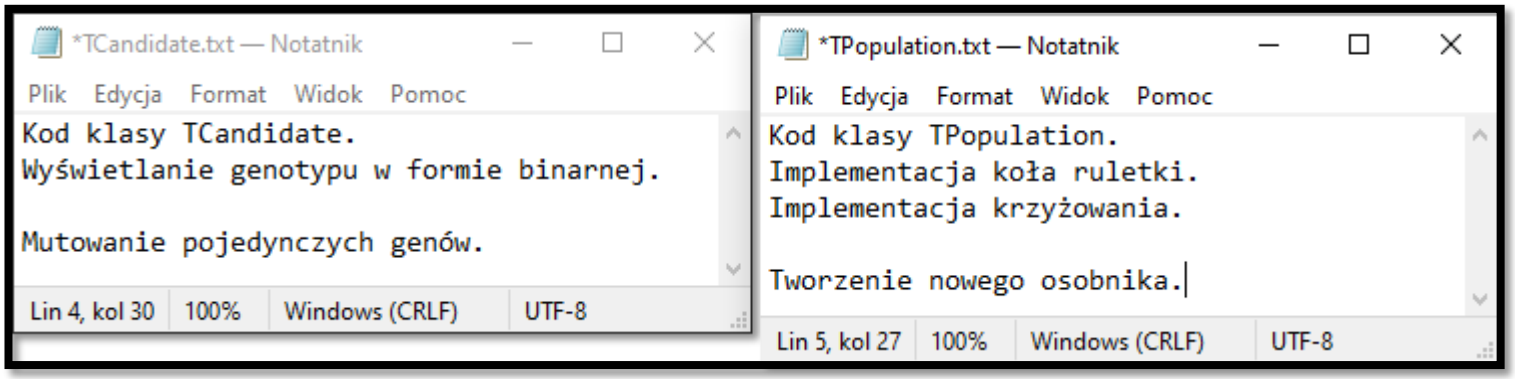




>> git commit –a –m „Implementacja koła ruletki”



>> git commit –a –m
„Implementacja krzyżowania”



>> git commit –a –m
„Implementacja mutowania”

*TAlgorithm.txt — No...

Plik Edycja Format Widok Pomoc

```
Kod klasy TAlgorithm.

Warunki zatrzymania:
-> max liczba populacji: 3

Tworzenie nowej populacji.
```

100% Windows (CRLF) UTF-8

>> git commit –a –m „Tworzenie nowej populacji”

>> git log --oneline

```
a0fcd81 (HEAD -> master) Tworzenie nowej populacji
a78e5ed Implementacja mutowania
63678c0 Implementacja krzyżowania
ad851c0 Implementacja koła ruletki
114d9e3 Klasa TAlgorithm
ecc6a73 Klasa TPopulation
0f73f90 Klasa TCandidate
e02b299 Klasa TGen
```

Na tym etapie posiadamy zaimplementowaną podstawową funkcjonalność algorytmu genetycznego.

Wszelkie modyfikacje kodu będą polegały na udoskonaleniu obecnie działającego rozwiązania. Naprawianie błędów i, przede wszystkim, wprowadzanie nowych funkcjonalności będzie się odbywać najprawdopodobniej równolegle – przez nas samych lub współautorów projektu.

Tworzenie nowych gałęzi

Modyfikacja 1: poszukiwanie minimum funkcji

Załóżmy, że jedną z pierwszych modyfikacji było poszukiwanie minimalnej wartości funkcji opisującej rozwiązywany problem. Cały algorytm został napisany w taki sposób, aby premiować osobników o najwyższych wartościach funkcji celu.

Funkcja celu może przyjąć różną wartość:

- $funkcja_{celu} = \frac{1}{funkcja\ opisująca\ problem}$;
- $funkcja_{celu} = -funkcja\ opisująca\ problem$

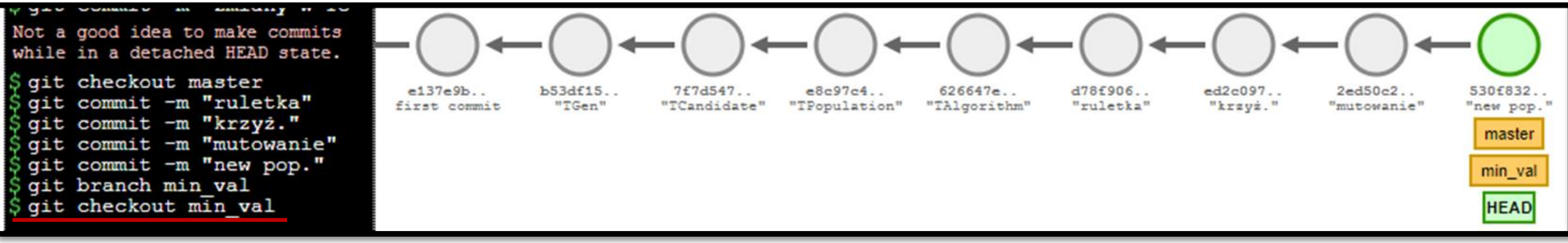
Obie powyższe funkcje celu mogą zostać zaimplementowane – od ich postaci zależy szybkość uzyskiwania zbieżności (przez rozrzut wartości oceny osobników). Jest więc co testować…

Aby nie tworzyć bałaganu w projekcie (mamy w końcu działający algorytm, dostępny dla wszystkich współpracowników w ostatnim *commit*), tworzymy odnogę w projekcie – **<u>gałąź</u>**. To w nowo utworzonej gałęzi będziemy prowadzić testy.

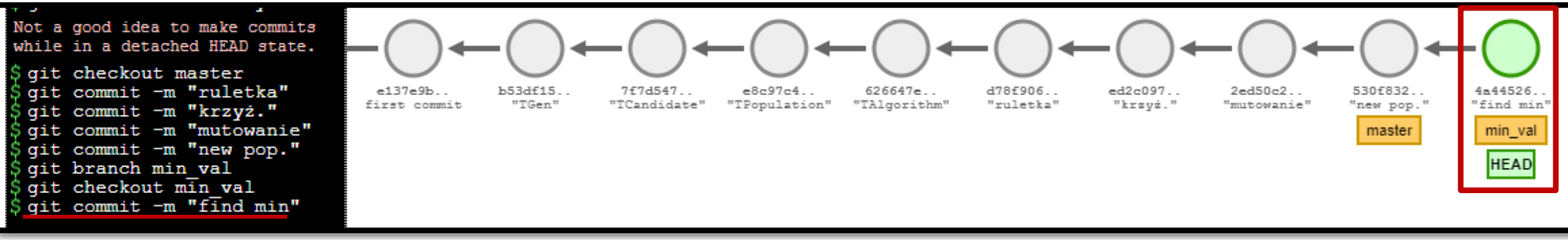
```
Have fun.
$ git commit -m "TGen"
$ git commit -m "TCandidate"
$ git commit -m "TPopulation"
$ git commit -m "TAlgorithm"
$ git checkout 7f7d457
Cannot find commit: 7f7d457
$ git checkout 7f7d547
$ git commit -m "zmiany w TC"
Not a good idea to make commits
while in a detached HEAD state.
$ git checkout master
$ git commit -m "ruletka"
$ git commit -m "krzyż."
$ git commit -m "mutowanie"
$ git commit -m "new pop."
$ git branch min_val
```

Nową gałąź tworzymy za pomocą komendy „git branch <*nazwa_gałęzi*>”

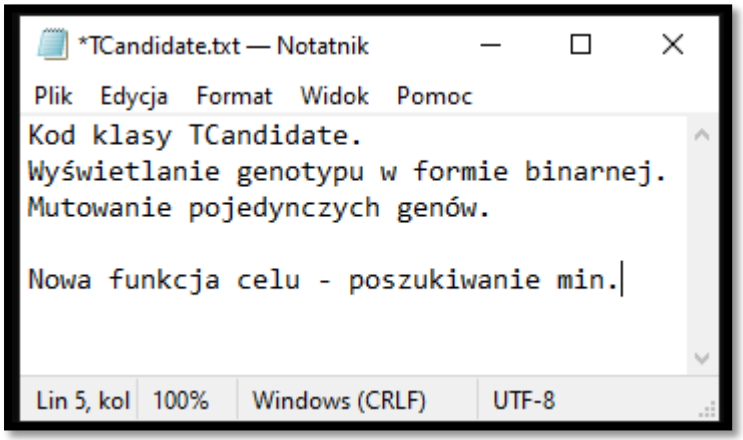
Teraz musimy przejść do nowej gałęzi, aby to w niej wprowadzać modyfikacje w kodzie. Robimy to za pomocą komendy „git checkout <*nazwa_gałęzi*>”.



Od teraz commit będzie powiązany z nowo utworzoną gałęzią.



>> git branch min_val
>> git checkout min_val



>> git commit –a –m „Nowa funkcja celu – poszukiwanie minimum”
>> git log --oneline

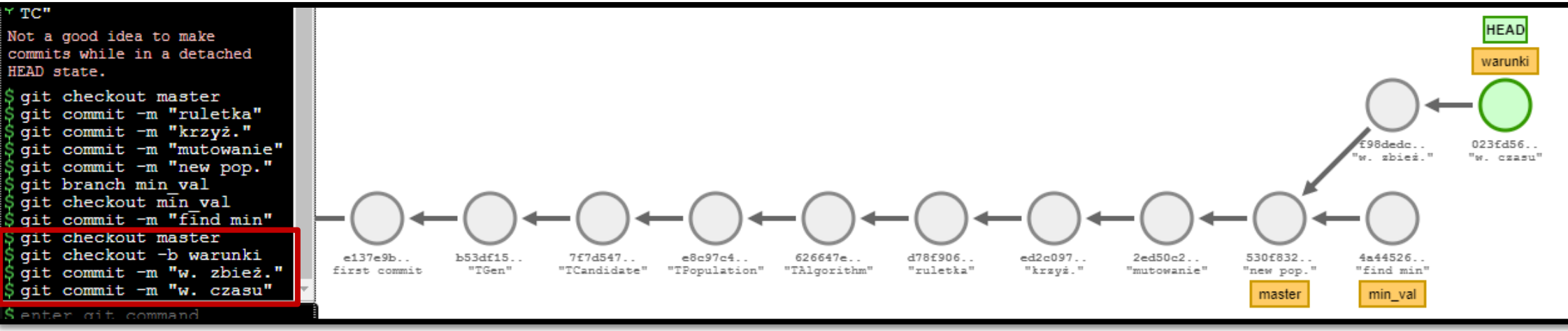
```
824e303 (HEAD -> min_val) Nowa funkcja celu - poszukiwanie minimum
a0fcd81 (master) Tworzenie nowej populacji
a78e5ed Implementacja mutowania
63678c0 Implementacja krzyżowania
ad851c0 Implementacja koła ruletki
114d9e3 Klasa TAlgorithm
ecc6a73 Klasa TPopulation
0f73f90 Klasa TCandidate
e02b299 Klasa TGen
```

Modyfikacja 2: rozbudowa warunku zatrzymania

Załóżmy, że równolegle my lub współpracownik pracujemy nad poprawą warunku zatrzymania algorytmu. Wprowadzamy po kolei dwa nowe warunki zatrzymania:

- warunek zbieżności – jeśli najlepszy wynik z *N* kolejnych populacji się nie różni o więcej niż *X%* zatrzymujemy algorytm i wyświetlamy najlepszy wynik;
- warunek czasu – jeśli obliczenia trwają dłużej niż *Y* godzin czekamy na zakończenie oceny osobników z obecnej populacji, wychodzimy z pętli i wyświetlamy najlepszy wynik.

Test nowej funkcjonalności wymaga nowej gałęzi…



>> git checkout –b warunki

=

>> git branch warunki
>> git checkout warunki

- git checkout master → wracamy do głównej gałęzi, chcemy działać na najprostszej działającej wersji algorytmu, nie zawierającej testów nowych funkcjonalności (np. z gałęzi *min_val*),
- git checkout –b warunki → tworzymy i przechodzimy do nowej gałęzi *warunki*, naszej przestrzeni do testów warunków zatrzymania algorytmu genetycznego,
- git commit … → testujemy nowe metody zatrzymania algorytmu genetycznego, wszystko w nowej, dedykowanej gałęzi

>> git checkout –b warunki

>> git log --oneline

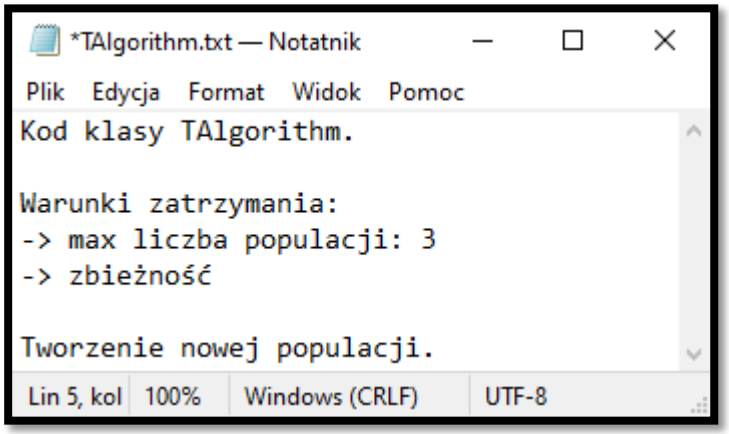
```
a0fcd81 (HEAD -> warunki, master) Tworzenie nowej populacji
a78e5ed Implementacja mutowania
63678c0 Implementacja krzyżowania
ad851c0 Implementacja koła ruletki
114d9e3 Klasa TAlgorithm
ecc6a73 Klasa TPopulation
0f73f90 Klasa TCandidate
e02b299 Klasa TGen
```

Log wyświetla informacje
o stanie projektu w danej gałęzi

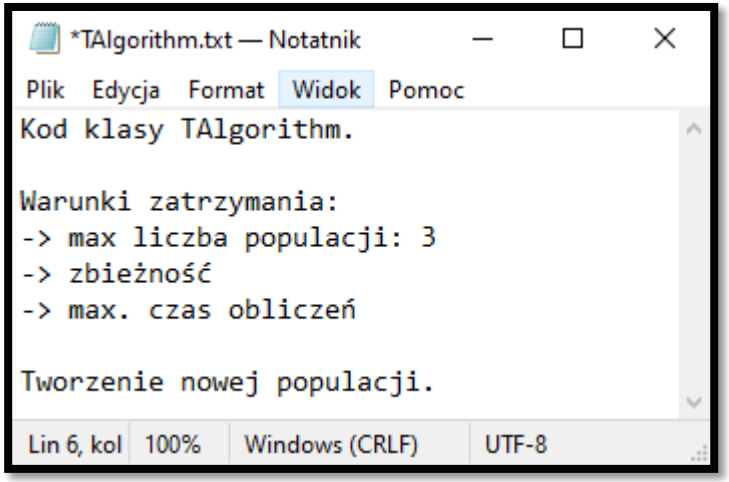
>> git branch

```
  master
  min_val
* warunki
```

Git branch informuje nas o wszystkich istniejących w repozytorium gałęziach (w tym *min_val*) i o tym, na której gałęzi się znajdujemy



>> git commit –a –m „Dodanie warunku zbieżności”



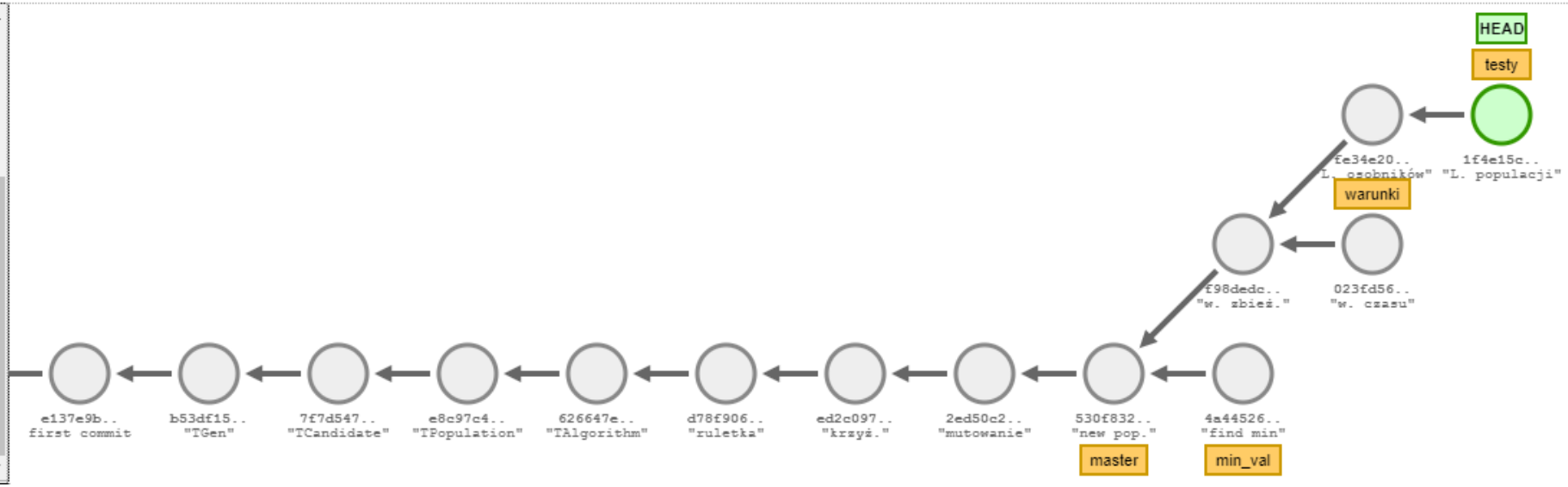
>> git commit –a –m „Dodanie warunku max. czasu obliczeń”

Modyfikacja 3: dobranie liczby osobników w populacji

W trakcie wprowadzania dwóch pierwszych modyfikacji okazało się, że zajmują one zbyt dużo czasu, a termin oddania projektu się zbliża… Warunek zbieżności został już zaimplementowany (jest on bardzo ważny z punktu widzenia stabilności/powtarzalności działania algorytmu genetycznego), co pozwoliło na przeprowadzenie wstępnych testów nad doborem liczby osobników w populacji.

Test wymaga nowej gałęzi…

```
$ git commit -m "zmiany w TC"
Not a good idea to make commits while in a detached HEAD state.
$ git checkout master
$ git commit -m "ruletka"
$ git commit -m "krzyż."
$ git commit -m "mutowanie"
$ git commit -m "new pop."
$ git branch min_val
$ git checkout min_val
$ git commit -m "find min"
$ git checkout master
$ git checkout -b warunki
$ git commit -m "w. zbież."
$ git commit -m "w. czasu"
$ git checkout f98dedc
$ git checkout -b testy
$ git commit -m "L. osobników"
$ git commit -m "L. populacji"
```



>> git log --online

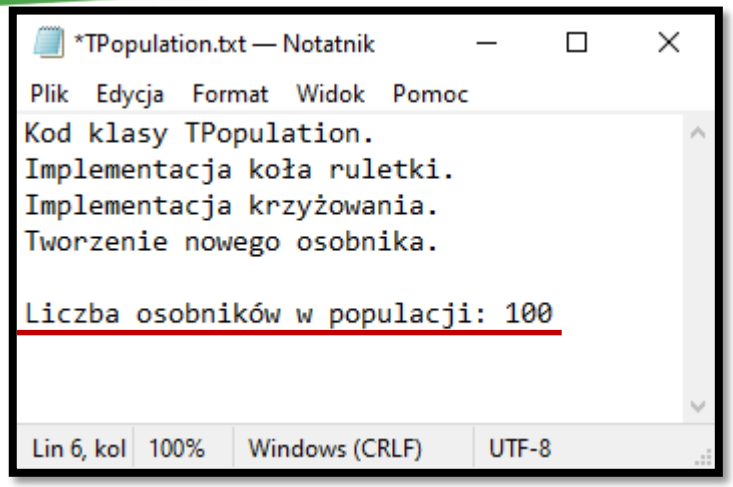
```
faa4b07 (HEAD -> warunki) Dodanie warunku max. czasu obliczeń
efef7f7 Dodanie warunku zbieżności
a0fcd81 (master) Tworzenie nowej populacji
a78e5ed Implementacja mutowania
63678c0 Implementacja krzyżowania
ad851c0 Implementacja koła ruletki
114d9e3 Klasa TAlgorithm
ecc6a73 Klasa TPopulation
0f73f90 Klasa TCandidate
e02b299 Klasa TGen
```

>> git checkout efef7f7

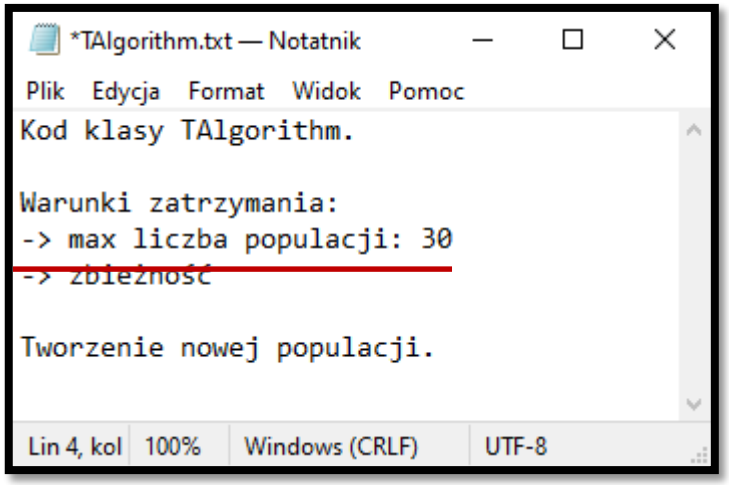
>> git checkout –b testy

>> git log --online

```
efef7f7 (HEAD -> testy) Dodanie warunku zbieżności
a0fcd81 (master) Tworzenie nowej populacji
a78e5ed Implementacja mutowania
63678c0 Implementacja krzyżowania
ad851c0 Implementacja koła ruletki
114d9e3 Klasa TAlgorithm
ecc6a73 Klasa TPopulation
0f73f90 Klasa TCandidate
e02b299 Klasa TGen
```



>> git commit –a –m „Testy: liczba osobników w populacji”

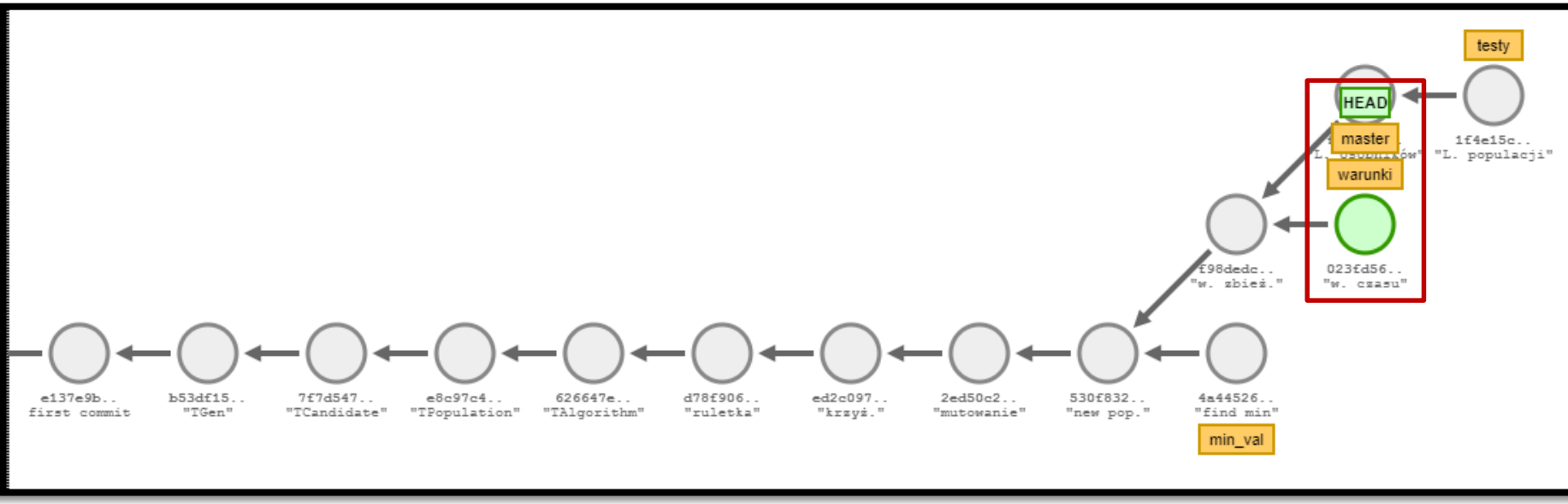


>> git commit –a –m „Testy: max. liczba populacji”

Łączenie gałęzi

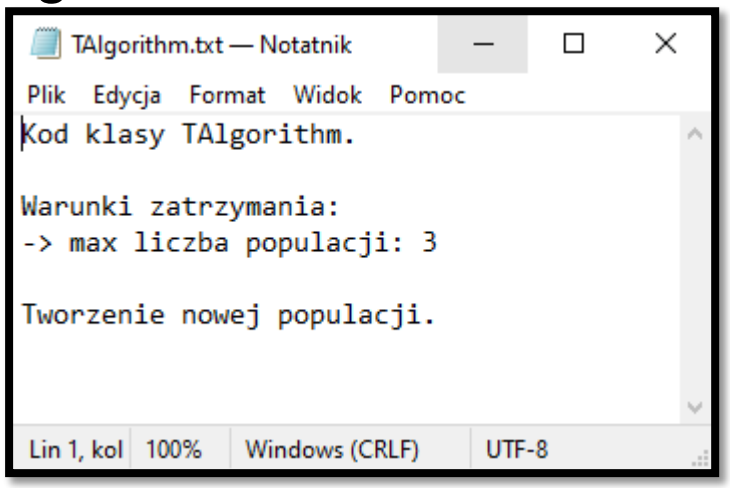
Udało się dokończyć pracę nad warunkami zatrzymania algorytmu. Zmiany można już wprowadzić do głównej gałęzi projektu.

```
while in a detached HEAD state.
$ git checkout master
$ git commit -m "ruletka"
$ git commit -m "krzyż."
$ git commit -m "mutowanie"
$ git commit -m "new pop."
$ git branch min_val
$ git checkout min_val
$ git commit -m "find min"
$ git checkout master
$ git checkout -b warunki
$ git commit -m "w. zbież."
$ git commit -m "w. czasu"
$ git checkout f98dedc
$ git checkout -b testy
$ git commit -m "L.
$ osobników"
$ git commit -m "L.
$ populacji"
$ git checkout master
$ git merge warunki
You have performed a fast-forward
merge.
```



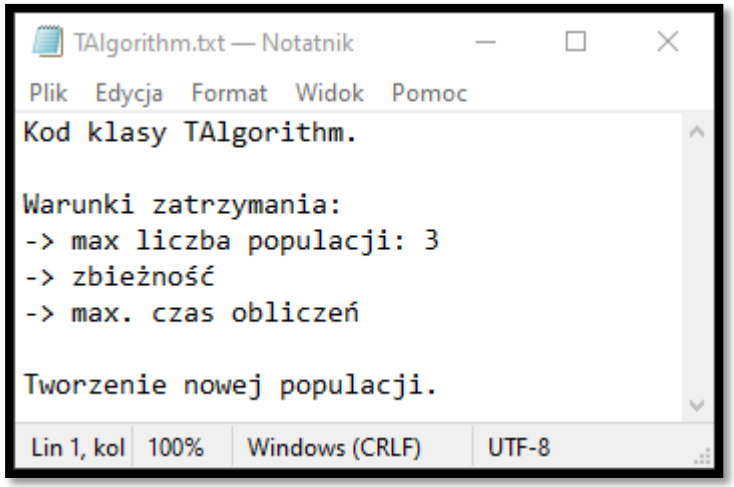
1. Przechodzimy na główną gałąź.
2. Wywołujemy komendę *merge* wskazując na odpowiednią gałąź

>> git checkout master



Podglądamy stan pliku Talgorithm.txt w gałęzi *master*

>> git merge warunki



Plik został zaktualizowany do postaci z gałęzi *warunki*

Symulacja

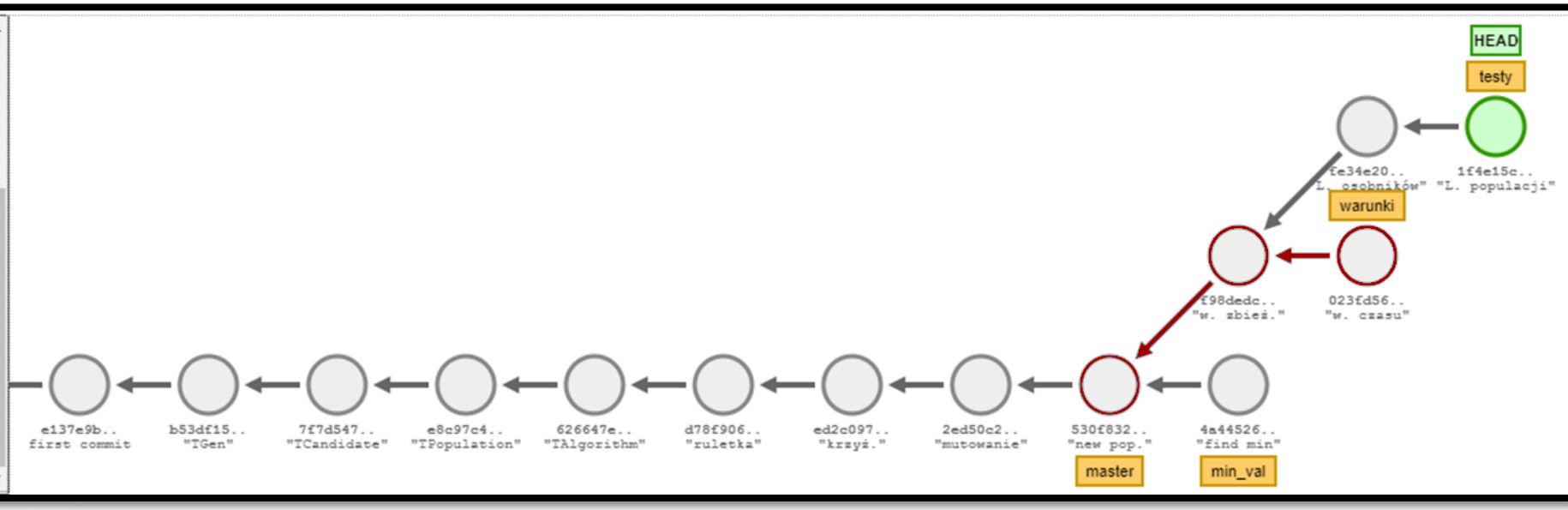
```
$ git checkout master
$ git merge warunki
You have performed a fast-forward
merge.
```

Konsola

```
Updating a0fcd81..faa4b07
Fast-forward
 TAlgorithm.txt | 2 ++
 1 file changed, 2 insertions(+)
```

Commity z gałęzi *warunki* bezpośrednio dziedziczą po gałęzi *master*. Dzięki temu możliwym było przeprowadzenie najłatwiejszej metody łączenia gałęzi – *fast-forward*.

```
$ git commit -m "zmiany w
TC"
Not a good idea to make
commits while in a detached
HEAD state.
$ git checkout master
$ git commit -m "ruletka"
$ git commit -m "krzyż."
$ git commit -m "mutowanie"
$ git commit -m "new pop."
$ git branch min_val
$ git checkout min_val
$ git commit -m "find min"
$ git checkout master
$ git checkout -b warunki
$ git commit -m "w. zbież."
$ git commit -m "w. czasu"
$ git checkout f98dedc
$ git checkout -b testy
$ git commit -m "L.
osobników"
$ git commit -m "L.
populacji"
```



Zakończyły się również testy nad doborem liczby osobników w populacji i warunkiem maksymalnej liczby populacji w algorytmie. Połączymy gałęzie *master* i *testy*.

>> git branch

```
* master
  min_val
  testy
  warunki
```

Upewniamy się, że jesteśmy na gałęzi *master*

>> git merge testy

```
Auto-merging TAlgorithm.txt
Merge made by the 'ort' strategy.
 TAlgorithm.txt   | 2 +-
 TPopulation.txt  | 3 ++-
 2 files changed, 3 insertions(+), 2 deletions(-)
```

Automatycznie złączono różne wersje pliku Talgorithm metodą *ort*.

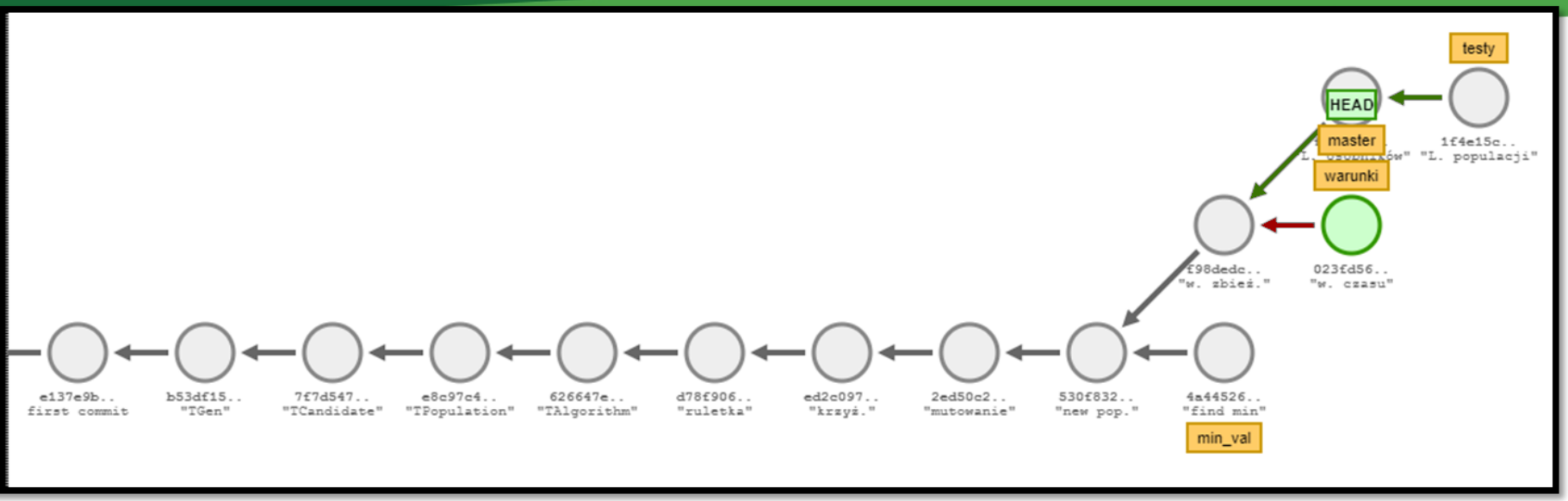
TAlgorithm.txt — Notatnik

Plik Edycja Format Widok Pomoc

```
Kod klasy TAlgorithm.

Warunki zatrzymania:
-> max liczba populacji: 30
-> zbieżność
-> max. czas obliczeń

Tworzenie nowej populacji.
```

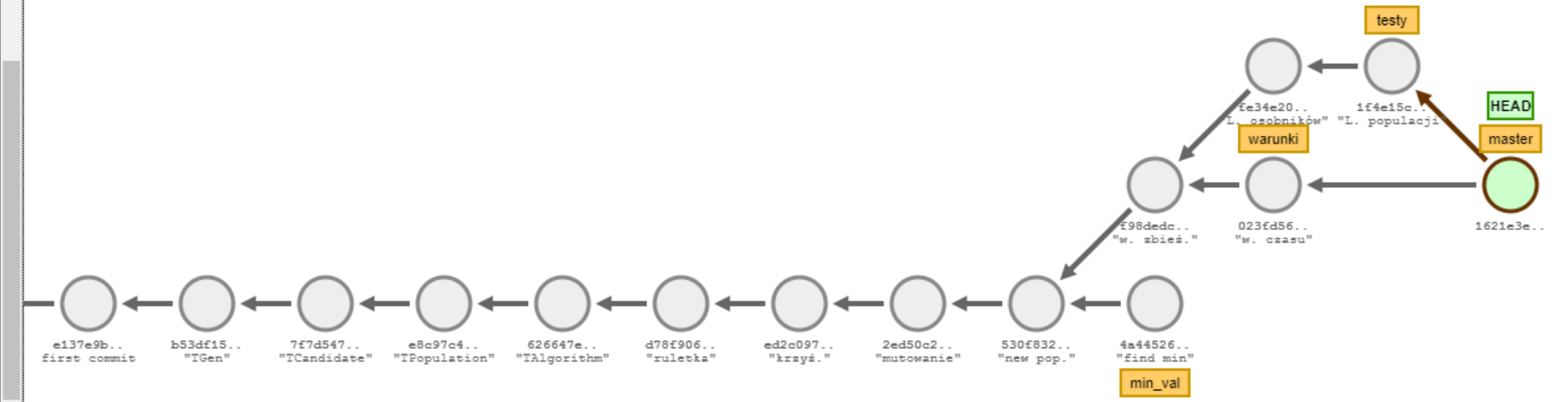
Lin 1, kol 100% Windows (CRLF) UTF-8

```
while in a detached HEAD state.
$ git checkout master
$ git commit -m "ruletka"
$ git commit -m "krzyż."
$ git commit -m "mutowanie"
$ git commit -m "new pop."
$ git branch min_val
$ git checkout min_val
$ git commit -m "find min"
$ git checkout master
$ git checkout -b warunki
$ git commit -m "w. zbież."
$ git commit -m "w. czasu"
$ git checkout f98dedc
$ git checkout -b testy
$ git commit -m "L.
osobników"
$ git commit -m "L.
populacji"
$ git checkout master
$ git merge warunki
You have performed a fast-forward
merge.
```



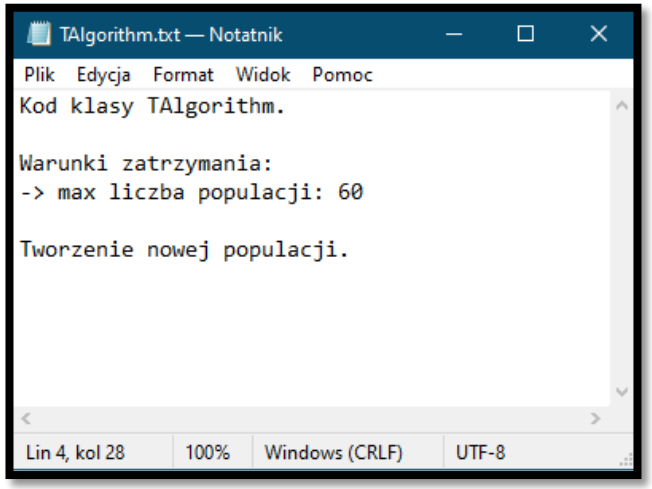
W tym przypadku *commity* z gałęzi *testy* nie dziedziczą już bezpośrednio po ostatnim *comicie* z gałęzi *master*. W takim przypadku należy rozwiązać ewentualne konflikty występujące w plikach – lub GIT spróbuje to zrobić za nas. *Commit* z etykietą *Head* posiada teraz dwójkę rodziców – czerwona strzałka wskazuje, gdzie doszło do „przeskoku”.

```
$ git commit -m "ruletka"
$ git commit -m "krzyż."
$ git commit -m "mutowanie"
$ git commit -m "new pop."
$ git branch min_val
$ git checkout min_val
$ git commit -m "find min"
$ git checkout master
$ git checkout -b warunki
$ git commit -m "w. zbież."
$ git commit -m "w. czasu"
$ git checkout f98dedc
$ git checkout -b testy
$ git commit -m "L.
osobników"
$ git commit -m "L.
populacji"
$ git checkout master
$ git merge warunki
You have performed a fast-
forward merge.
$ git merge testy
```



Załóżmy, że osoba testująca poszukiwanie minimalnej wartości funkcji opisującej rozwiązywany problem zrobiła dodatkowe testy dla tego przypadku zmieniając liczbę populacji w algorytmie. Omówione testy przesłała w osobnym *commicie* i nalega na wdrożenie zmian do gałęzi master.

>> git checkout min_val



>> git commit –a –m „Testy dla szukanej minimalnej wartości”

Tym razem przeprowadzimy operację merge w konsoli **Git CMD**:

>> cd *<ścieżka do repozytorium>*
>> git checkout master
>> git merge min_val

```
Auto-merging TAlgorithm.txt
CONFLICT (content): Merge conflict in TAlgorithm.txt
Automatic merge failed; fix conflicts and then commit the result.
```

>> git status

```
On branch master
You have unmerged paths.
  (fix conflicts and run "git commit")
  (use "git merge --abort" to abort the merge)

Changes to be committed:
        modified:   TCandidate.txt

Unmerged paths:
  (use "git add <file>..." to mark resolution)
        both modified:   TAlgorithm.txt
```

>> notepad TAlgorithm.txt

Na gałęziach *master* (*Head*) i *min_val* istnieją różne wersje pliku. GIT nie miał odwagi podjąć za nas decyzji, która wersja jest poprawna.

TAlgorithm.txt — Notatnik

Plik Edycja Format Widok Pomoc

```
Kod klasy TAlgorithm.

Warunki zatrzymania:
<<<<<<< HEAD
-> max liczba populacji: 30
-> zbieżność
-> max. czas obliczeń
=======
-> max liczba populacji: 60
>>>>>>> min_val

Tworzenie nowej populacji.
```

Lin 1, kol | 100% | Windows (CRLF) | UTF-8

Poprawiamy plik:

*TAlgorithm.txt — Notatnik

Plik Edycja Format Widok Pomoc

```
Kod klasy TAlgorithm.

Warunki zatrzymania:
-> max liczba populacji: 60
-> zbieżność
-> max. czas obliczeń

Tworzenie nowej populacji.
```

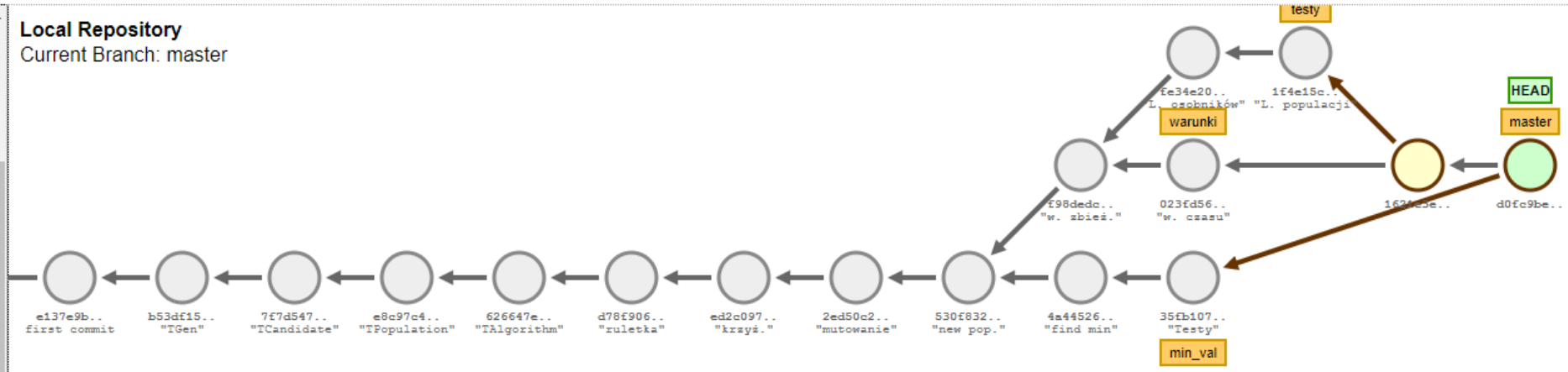
Lin 6, kol | 100% | Windows (CRLF) | UTF-8

>> git commit –a –m „Merge branch „min_val” ”

>> git log --oneline

```
7e59521 (HEAD -> master) Merge branch 'min_val'
f2d35a9 (min_val) Testy dla szukanej minimalnej wartości
1ce00fb Merge branch 'testy'
4176466 (testy) Testy: max. liczba populacji
c407a56 Testy: liczba osobników w populacji
faa4b07 (warunki) Dodanie warunku max. czasu obliczeń
efef7f7 Dodanie warunku zbieżności
824e303 Nowa funkcja celu - poszukiwanie minimum
a0fcd81 Tworzenie nowej populacji
a78e5ed Implementacja mutowania
63678c0 Implementacja krzyżowania
ad851c0 Implementacja koła ruletki
114d9e3 Klasa TAlgorithm
ecc6a73 Klasa TPopulation
0f73f90 Klasa TCandidate
e02b299 Klasa TGen
```

```
$ git checkout 7f7d547
$ git commit -m "zmiany w TC"
Not a good idea to make commits while in a detached HEAD state.
$ git checkout master
$ git commit -m "ruletka"
$ git commit -m "krzyż."
$ git commit -m "mutowanie"
$ git commit -m "new pop."
$ git branch min_val
$ git checkout min_val
$ git commit -m "find min"
$ git checkout master
$ git checkout -b warunki
$ git commit -m "w. zbież."
$ git commit -m "w. czasu"
$ git checkout f98dedc
$ git checkout -b testy
$ git commit -m "L. osobników"
$ git commit -m "L. populacji"
$ git checkout master
$ git merge warunki
You have performed a fast-forward merge.
$ git merge testy
$ git checkout min_val
$ git commit -m "Testy"
$ git checkout master
$ git merge min_val
```



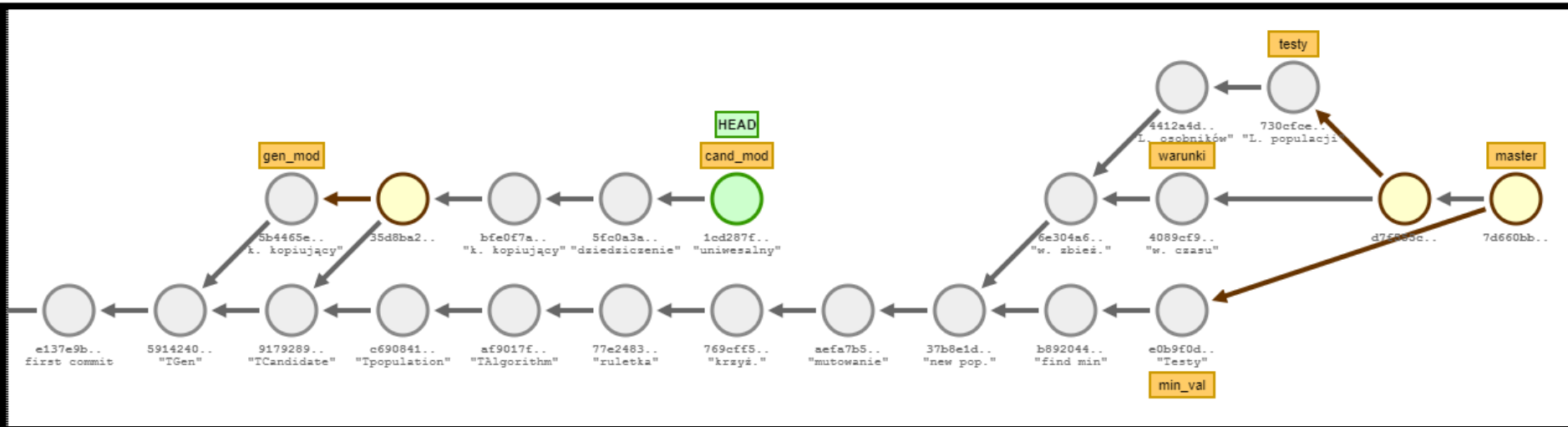
>> git log --graph --oneline

```
*   7e59521 (HEAD -> master) Merge branch 'min_val'
|\
| * f2d35a9 (min_val) Testy dla szukanej minimalnej wartości
| * 824e303 Nowa funkcja celu - poszukiwanie minimum
*   1ce00fb Merge branch 'testy'
|\ \
| * | 4176466 (testy) Testy: max. liczba populacji
| * | c407a56 Testy: liczba osobników w populacji
* | | faa4b07 (warunki) Dodanie warunku max. czasu obliczeń
|/ /
* / efef7f7 Dodanie warunku zbieżności
|/
* a0fcd81 Tworzenie nowej populacji
* a78e5ed Implementacja mutowania
* 63678c0 Implementacja krzyżowania
* ad851c0 Implementacja koła ruletki
* 114d9e3 Klasa TAlgorithm
* ecc6a73 Klasa TPopulation
* 0f73f90 Klasa TCandidate
* e02b299 Klasa TGen
```

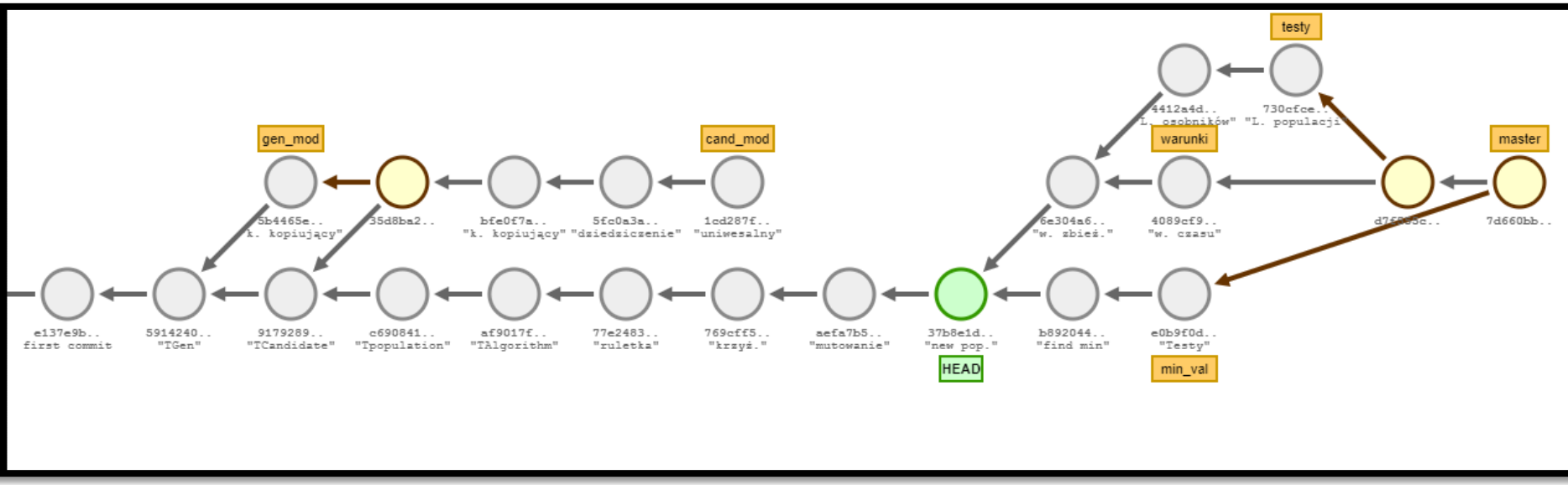
Przebudowa projektu
(część nieobowiązkowa)

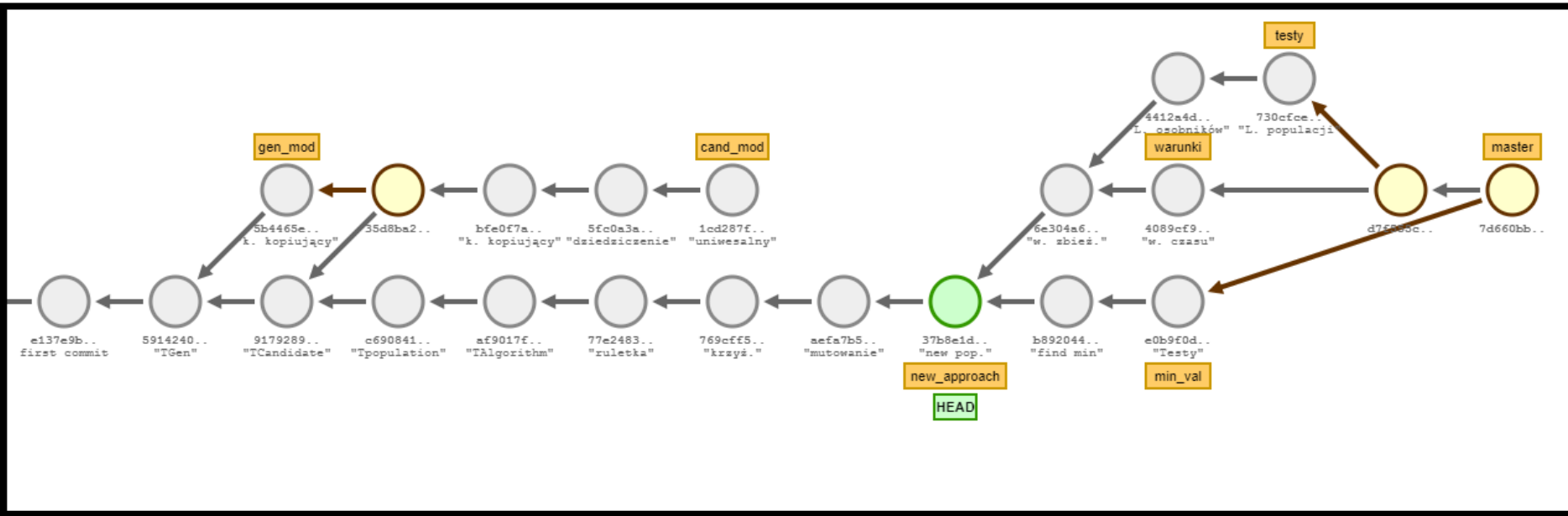
Wprowadzamy zmiany w klasach TGen i Tcandidate zgodnie z jednymi z ostatnich zajęć w semestrze (konstruktory kopiujące, dziedziczenie, klasy abstrakcyjne). Cofając się do jednych z pierwszych *commitów* nie występują błędy niezgodności deklaracji funkcji/klas w wyższych klasach, jak TPopulation i TAlgorithm.

Aby zobaczyć komendy należy skopiować rysunek po prawej

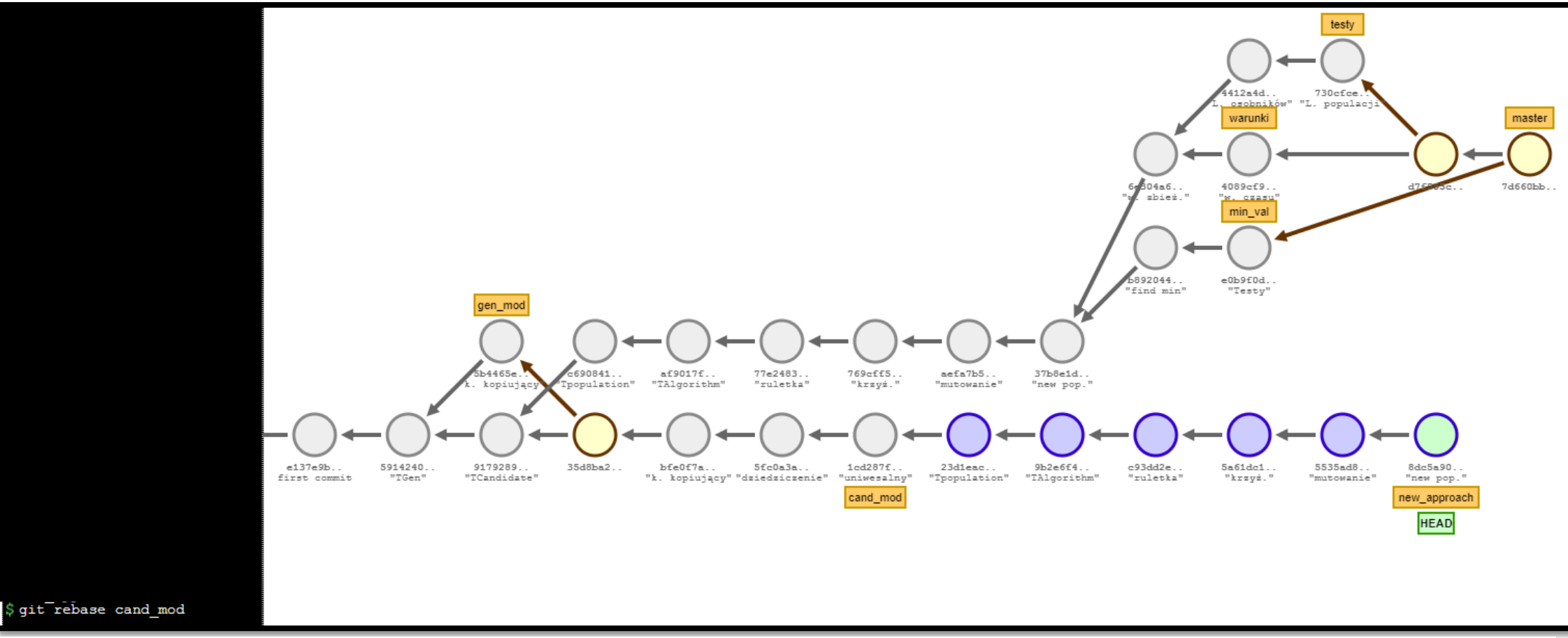


Przechodzimy do *commita*, gdzie algorytm genetyczny miał już swoją podstawową funkcjonalność i tworzymy nową gałąź.



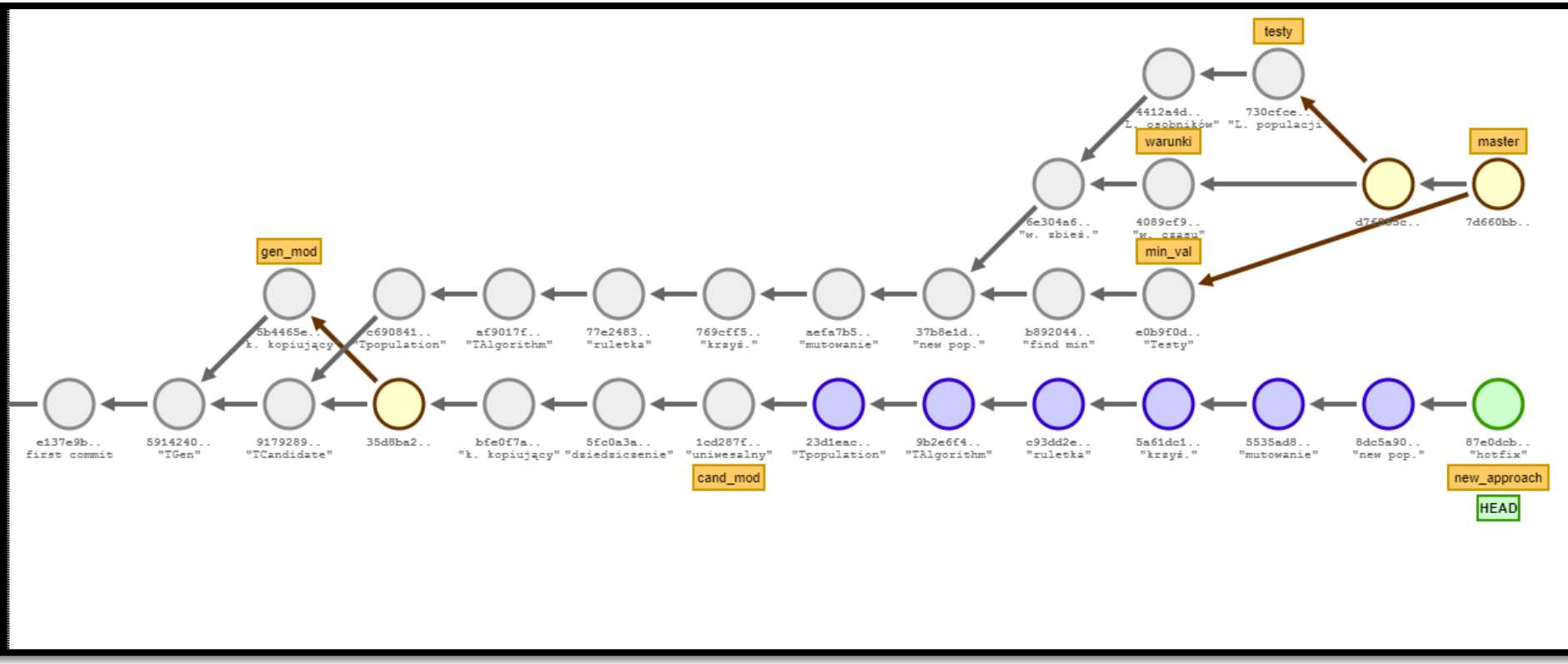


Za pomocą komendy *rebase* wskazujemy nową bazę / nowych rodziców dla aktywnego *commita*. Tym samym do obecnych wersji klas TPopulation i TAlgorithm oraz funkcjonalności koła ruletki, krzyżowania, mutowania i tworzenia nowej populacji zostały dodane nowe wersje klas TGen i TCandidate. Co istotne, zmiany te zostały wprowadzone do nowej gałęzi – cały czas mamy w odsieczy backup na głównej gałęzi *master*.

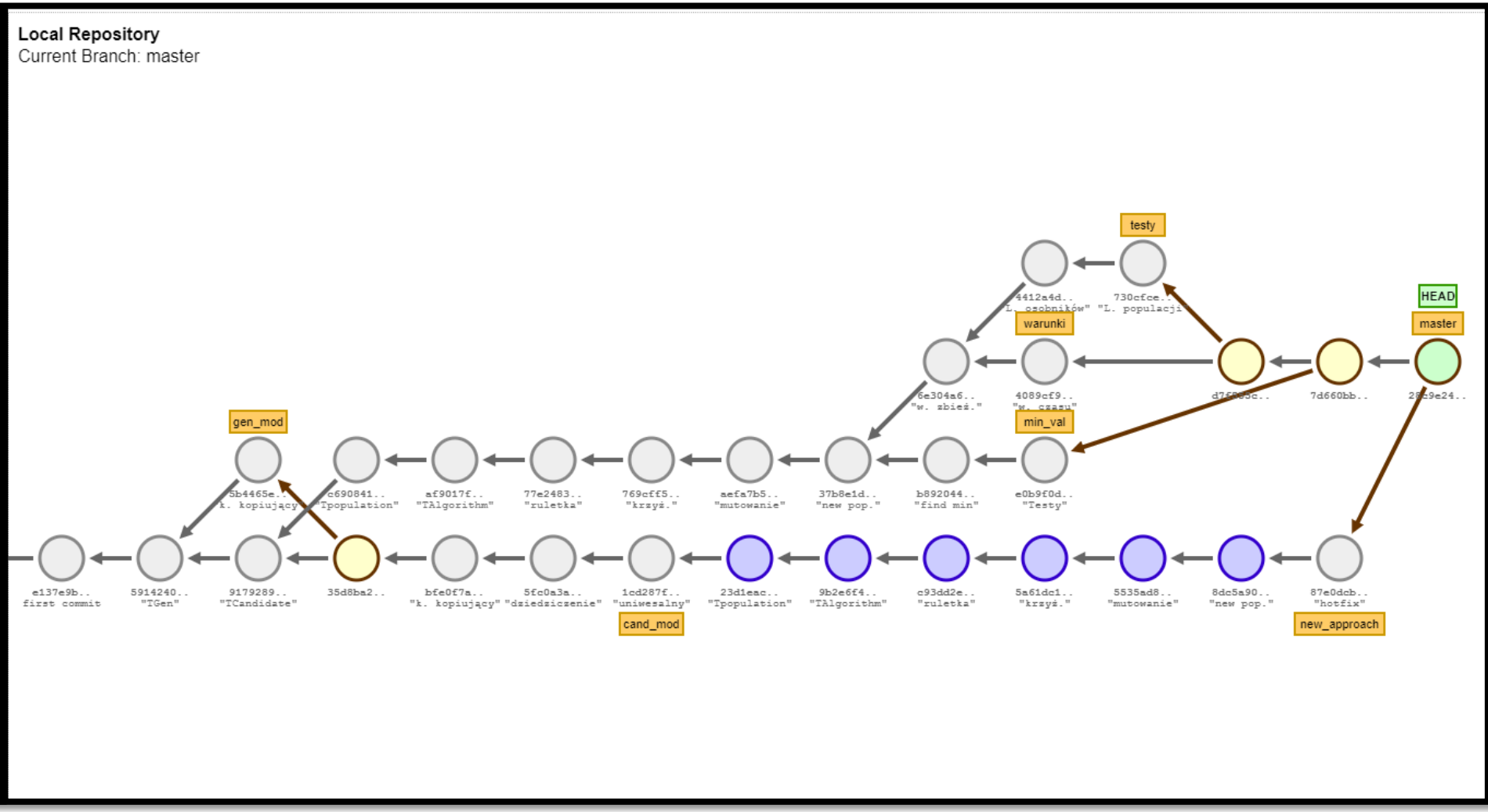


Pozostaje „tylko” zaktualizować klasy TPopulation i TAlgorithm, aby współpracowały w nowymi wersjami TGen i TCandidate.

Dla skutecznie naprawionych klas i funkcji tworzymy nowy *commit*.



Pozostaje wrzucić nową, bardziej uniwersalną, wersję algorytmu na gałąź *master*. Tym razem stosujemy zwykły *merge*, spodziewając się że modyfikacje omówione w tej instrukcji wymagają o wiele mniej pracy niż dostosowanie klas TPopulation i TAlgorithm.



git branch	Wyświetlenie informacji o istniejących gałęziach w repozytorium
git branch <*nazwa*>	Utworzenie gałęzi <*nazwa*>
git branch –d <*nazwa*>	Usunięcie gałęzi <*nazwa*>
git checkout <*nazwa*>	Przejście do gałęzi <*nazwa*> lub *commitu* o id <*nazwa*>
git checkout –b <*nazwa*>	Utworzenie i aktywowanie gałęzi <*nazwa*> - połączenie komend git branch <*nazwa*> + git checkout <*nazwa*>
git merge <*nazwa*>	Scalenie gałęzi: dodanie zmian z gałęzi <nazwa> do aktywnej gałęzi
git rebase <*nazwa*>	Dla aktywnego *commita* wskazanie i utworzenie nowej ścieżki rodziców – na podstawie gałęzi <*nazwa*>
git log --graph	*Graficzne pokazanie zależności między gałęziami i commitami dla aktywnego commita*